3.5.5 Conclusions

The analysis of the comparative performance and height of BSTs and randomized treaps suggests that, while the latter requires slightly more memory and can be slower in the generic case, when we don’t have any guarantee on the uniform distribution of keys nor on the order of the operations, using BSTs carries a far greater risk of becoming a bottleneck.

If you remember, when we introduced data structures in chapter 1 we made it clear: knowing the right data structure to use is more about avoiding the wrong choices than finding the perfect data structure; this is exactly the same case, we (as developers) need to be aware of the situations where we need to use a balanced tree to avoid attacks or just degraded performance.

It's worth reiterating that the first part of the analysis, focusing on the height of the trees, has general value[15] and is independent of the programming language used; the analysis of running time and memory usage, instead, only has value for this implementation, programming language, design choices etc…: all these aspects can, in theory, be optimized for your application’s specific requirements.

My advice, as always, is to carefully analyze requirements, understand what’s critical in your software and where you need certain guarantees about time and memory, and then test and profile the critical sections; be sure to avoid wasting time for non-critical sections, usually you’ll find that the Pareto principle holds for software and you can get an 80% performance gain by optimizing 20% of your code; although the exact ratio may vary, the overall principle that you can get a significant improvement by optimizing the most critical parts of your application will likely hold.